Machine learning: the basic ideas

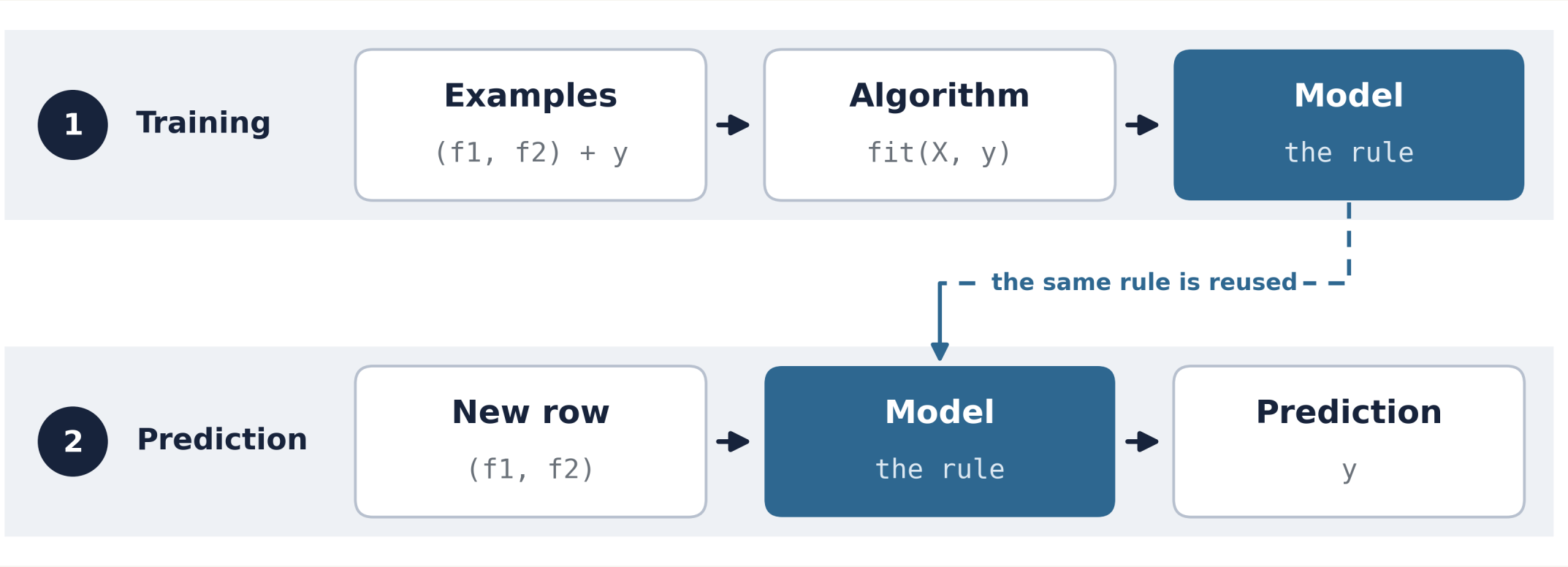
Lesson 2

Computational Neuroscience 2026/27 · Alan Perotti

06

Intuition

Learning is finding the rule



In classical programming we write the rule. In machine learning we give examples with their answers, and an algorithm finds the rule. The rule it finds is the model. Here it maps two measurements, f1 and f2, to a label y.

01

Example

Minimal ML pipeline

In [1]:

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

df = pd.read_csv('data.csv')
X, y = df[['f1', 'f2']], df['y']

model = RandomForestClassifier()
model.fit(X, y)
predictions = model.predict(X)

accuracy = accuracy_score(y, predictions)
print(f"Accuracy: {accuracy:.1%}")
```

```
Accuracy: 100.0%
```

Imports: the tools we bring in

```
import pandas as pd
```

pandas reads and handles tables. “as pd” gives it a short name.

```
from sklearn.ensemble import RandomForestClassifier
```

scikit-learn is the machine learning library. This line brings in the model we use: a random forest.

```
from sklearn.metrics import accuracy_score
```

A function that measures how many predictions are correct.

Data: from file to X and y

```
df = pd.read_csv('data.csv')
```

```
X, y = df[['f1', 'f2']], df['y']
```

Reads the file into a table (a DataFrame) with 1000 rows and the columns f1, f2 and y.

X holds the features, the two measurements. y holds the label to predict.

Model, fit, predict

```
model = RandomForestClassifier()
```

Creates the model. The hyperparameters have default values. Nothing is learned yet.

```
model.fit(X, y)
```

Learning: the parameters of the model are found from the examples X and their answers y.

```
predictions = model.predict(X)
```

The model answers for each row of X: one predicted label per row.

Score: how often is it right?

```
accuracy = accuracy_score(y, predictions)
```

```
print(f"Accuracy: {accuracy:.1%}")
```

Compares the true labels y with the predictions: the fraction of correct answers, from 0 to 1.

Prints it as a percentage. In the f-string, :.1% multiplies by 100, keeps one decimal and adds the % sign.

02

Data

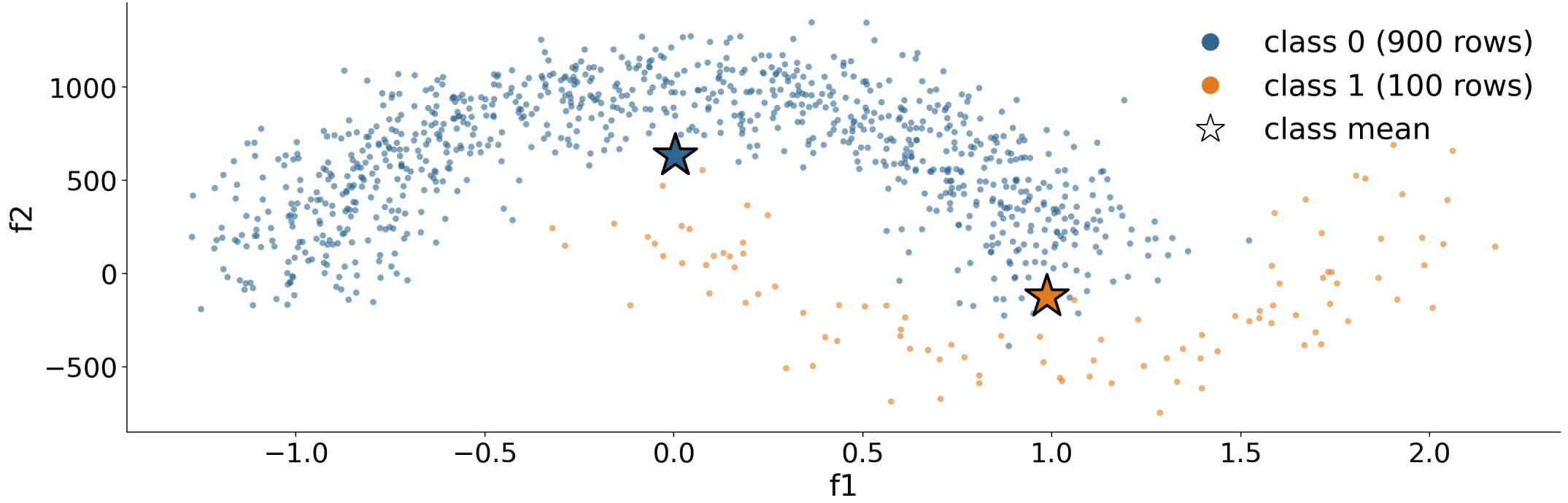
Data is a matrix: look at it first

In [2]:

```
print(X.shape)
print(y.value_counts())
print(X.agg(['mean', 'std']))
```

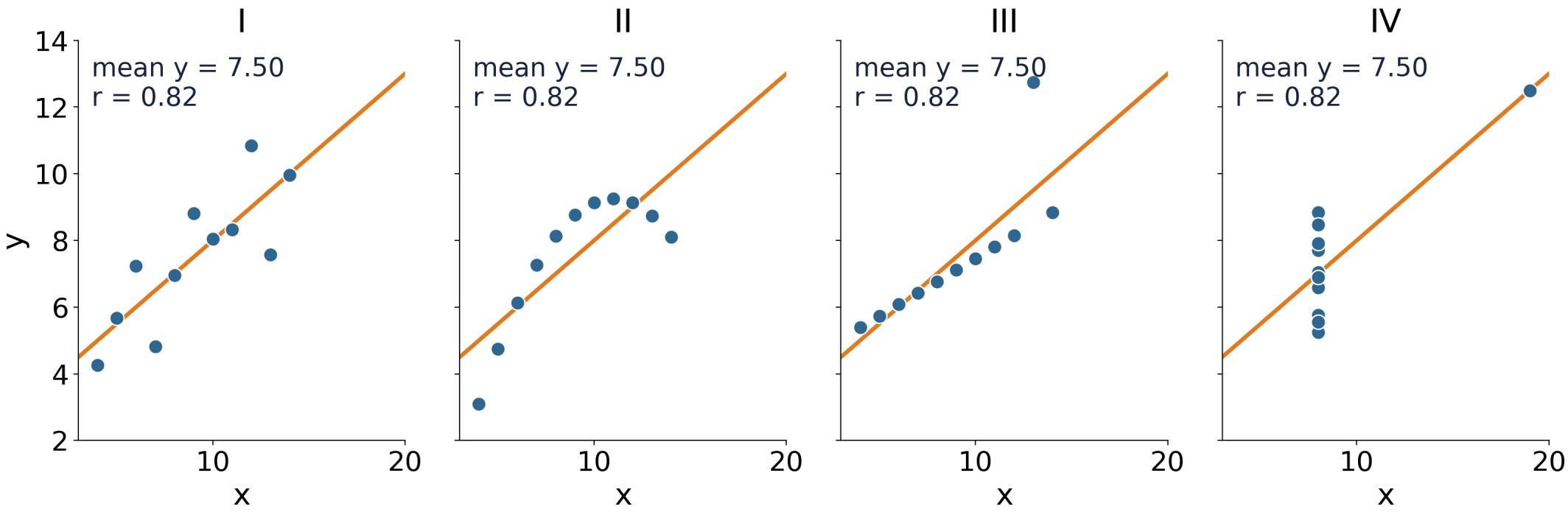
```
(1000, 2)
y
0    900
1    100
Name: count, dtype: int64
             f1           f2
mean   0.101832   557.412720
std    0.775385   412.426584
```

The data: a scatter plot



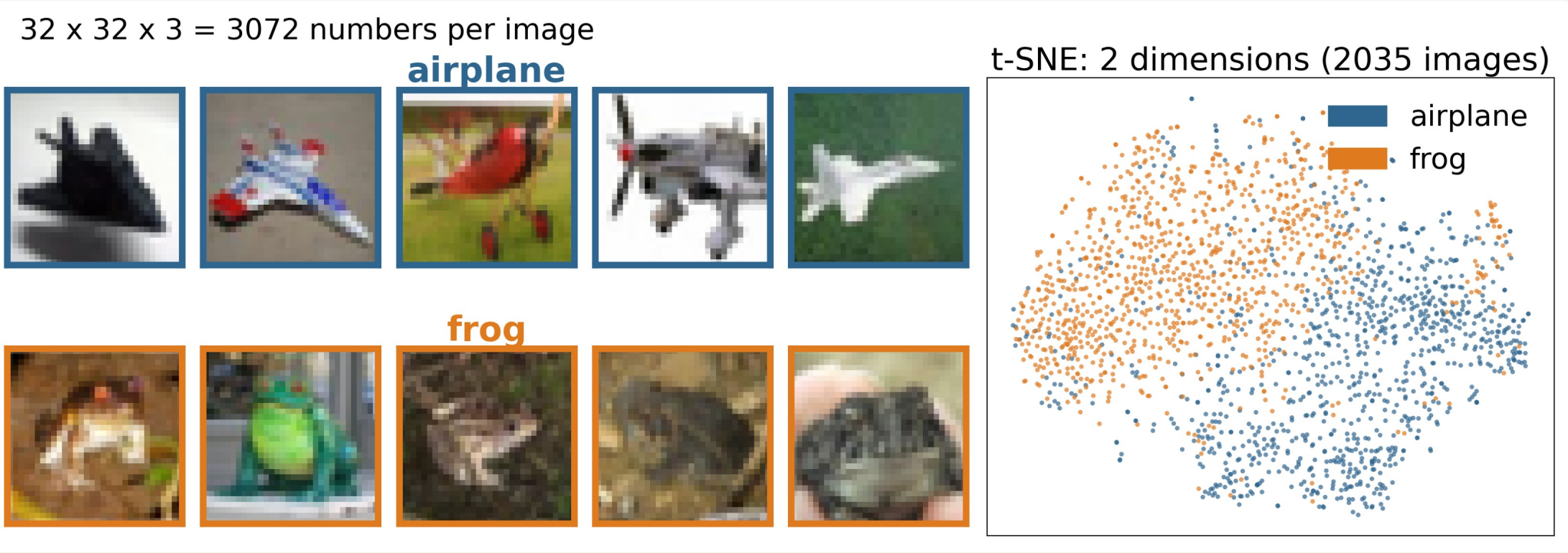
Each point is a row: f1 on the horizontal axis, f2 on the vertical axis, colour is the label. The stars mark the mean point of each class. The classes overlap near their border: that is noise.

Anscombe’s quartet: always plot



Four datasets with the same mean, the same variance, the same correlation and the same regression line. Their plots are completely different: noisy points, a curve, an outlier, a single point that decides the line. Numbers summarise, a plot shows. Visualise the data before modelling.

Many dimensions: t-SNE and UMAP



Real data often has many features: each CIFAR-10 image has 32 × 32 × 3 = 3072 numbers, and we cannot plot 3072 dimensions. t-SNE and UMAP map every row to 2 dimensions so that rows that are close in the original space stay close. Use them to look at the data, not to measure it: distances and cluster sizes in the plot are not reliable.

03

Model

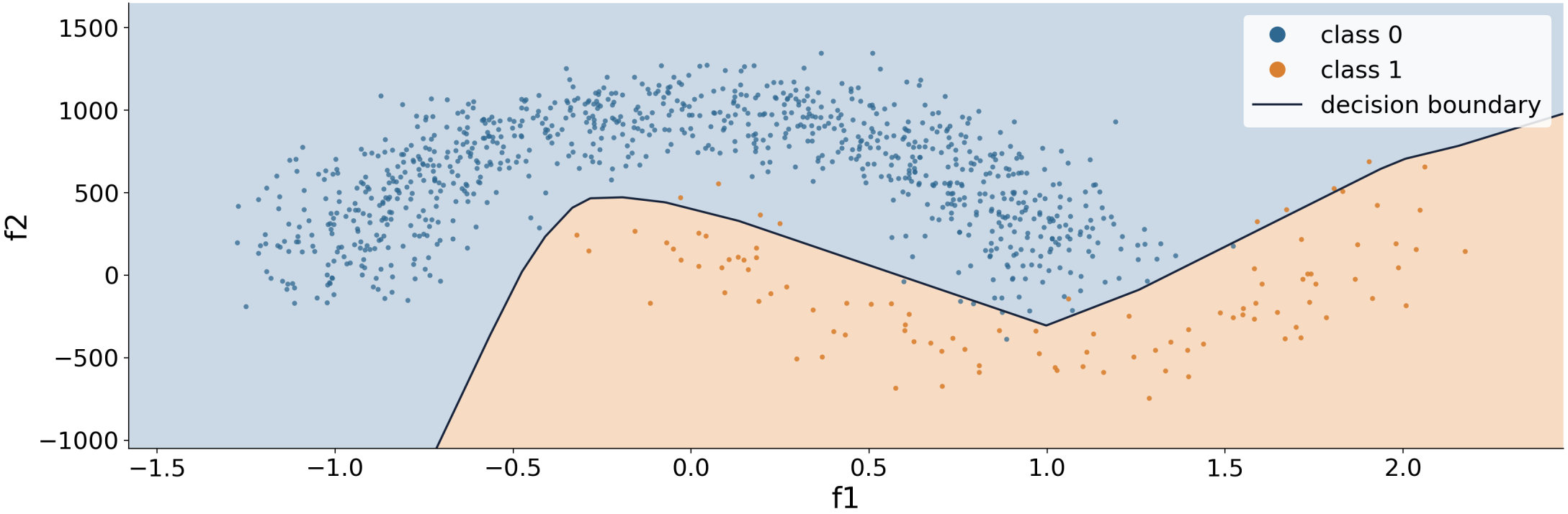
The model: a random forest

```
model = RandomForestClassifier()
```

What IS a random forest?

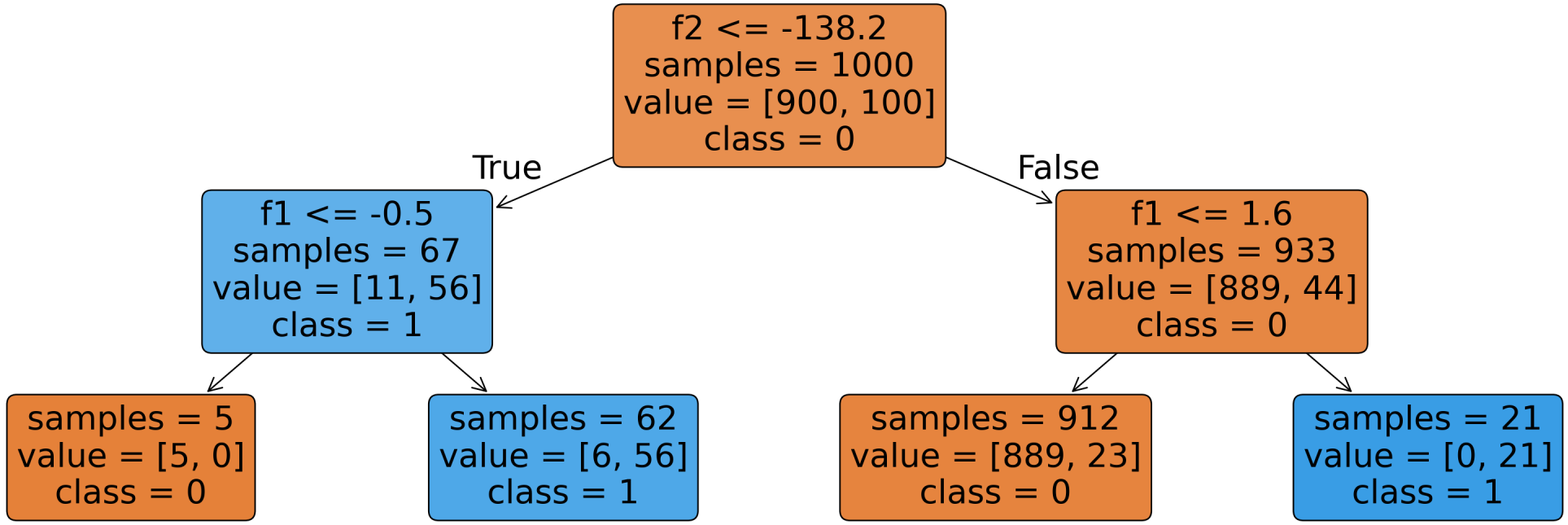
Why is the model a random forest?

Decision boundary



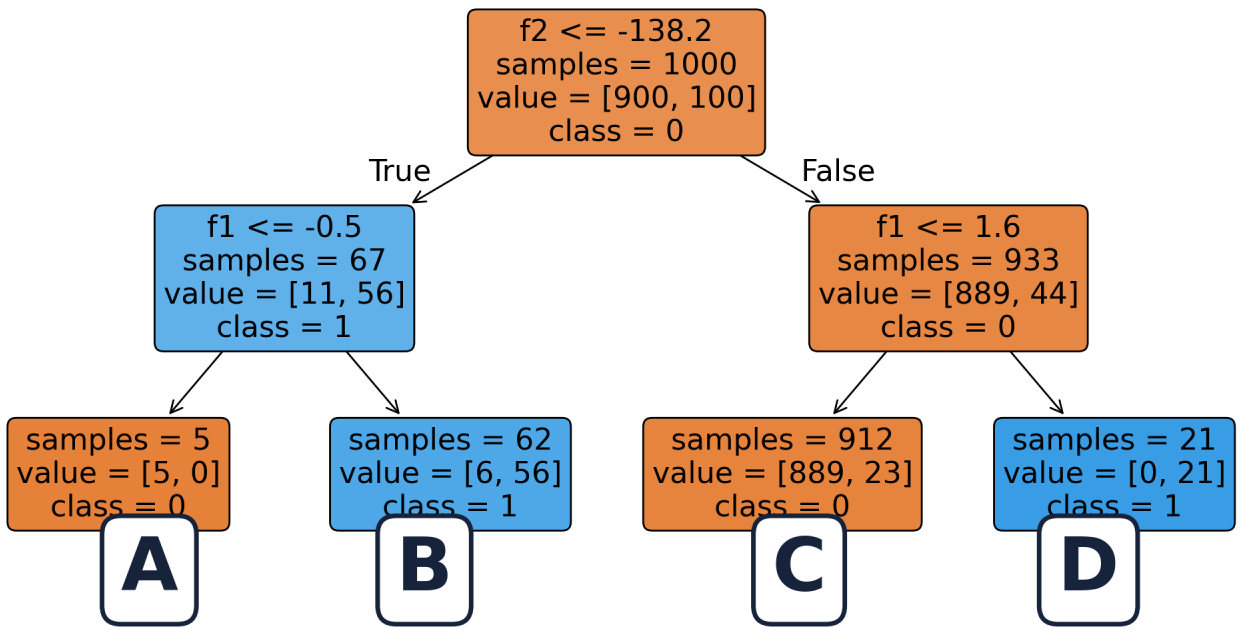
A classifier assigns a class to every point of the plane. The border between the two regions is the decision boundary. Here: an MLP trained on f1 and f2; the dots are the training rows.

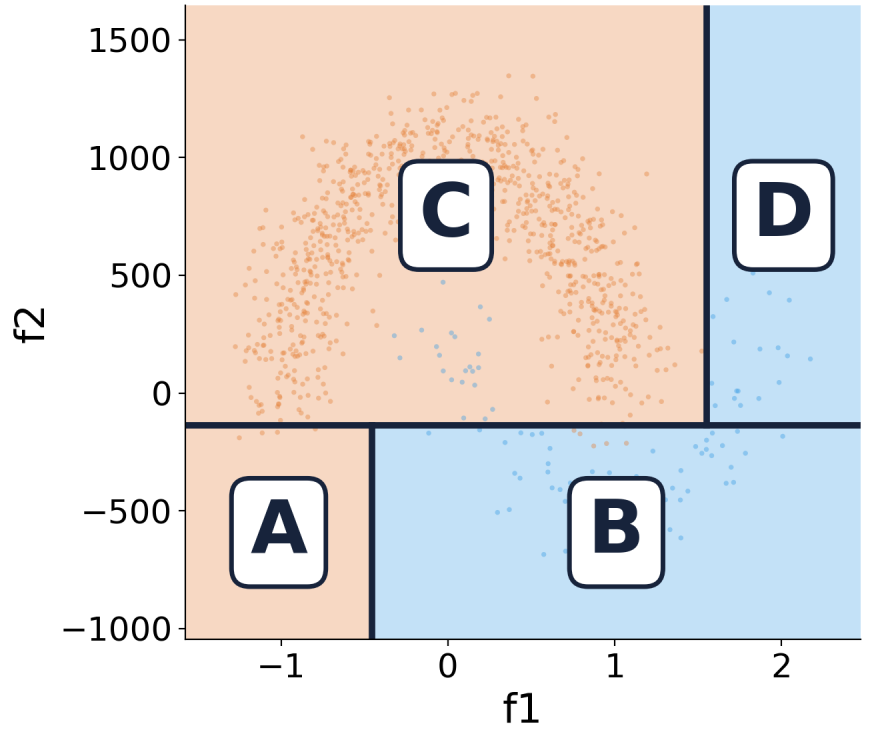
Model, parameters, hyperparameters



A model is a family of rules. Fitting picks one rule using the data. The learned values, such as the thresholds of this tree, are the parameters. The hyperparameters are chosen by us before fitting, such as the maximum depth of the tree.

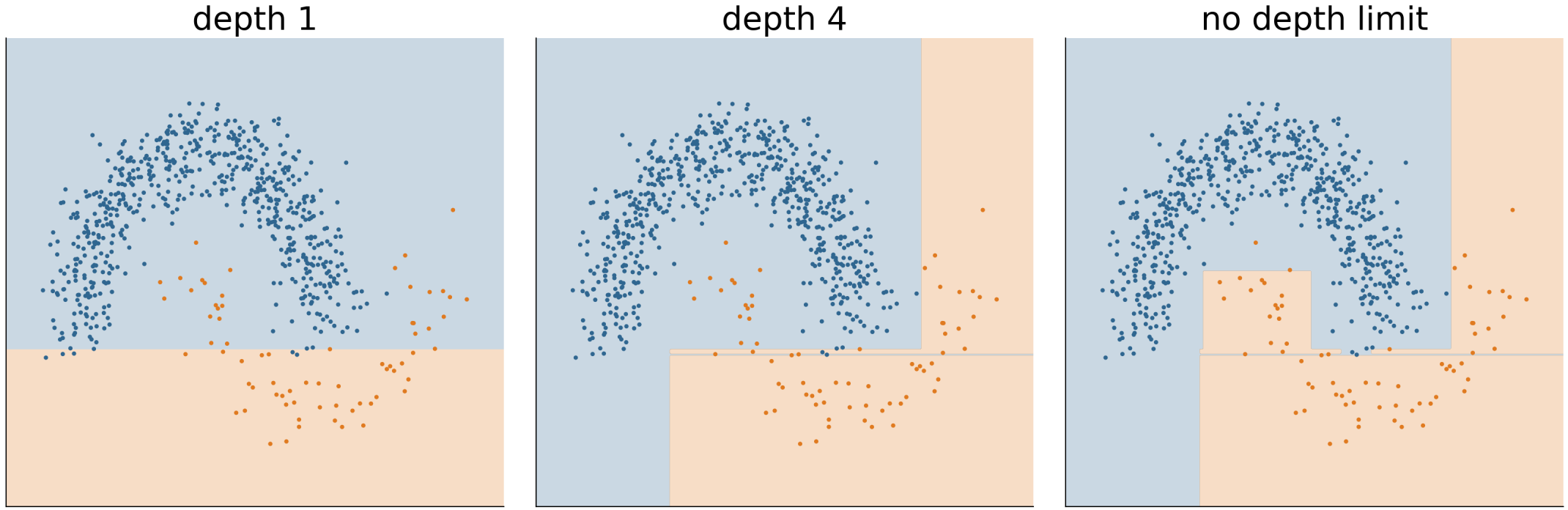
Decision tree: questions and regions





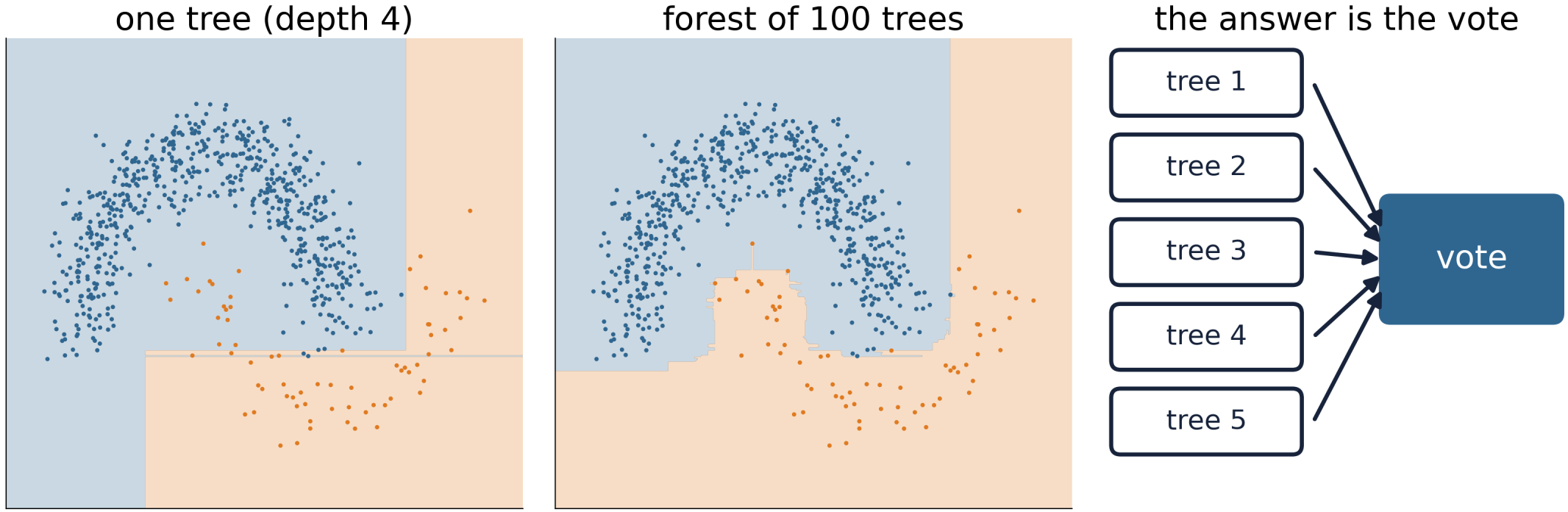
Each question splits the space with a vertical or horizontal line. Each leaf (A to D) is a rectangle of the feature space with a predicted class: the regions in the plot are the leaves of the tree.

Decision boundary



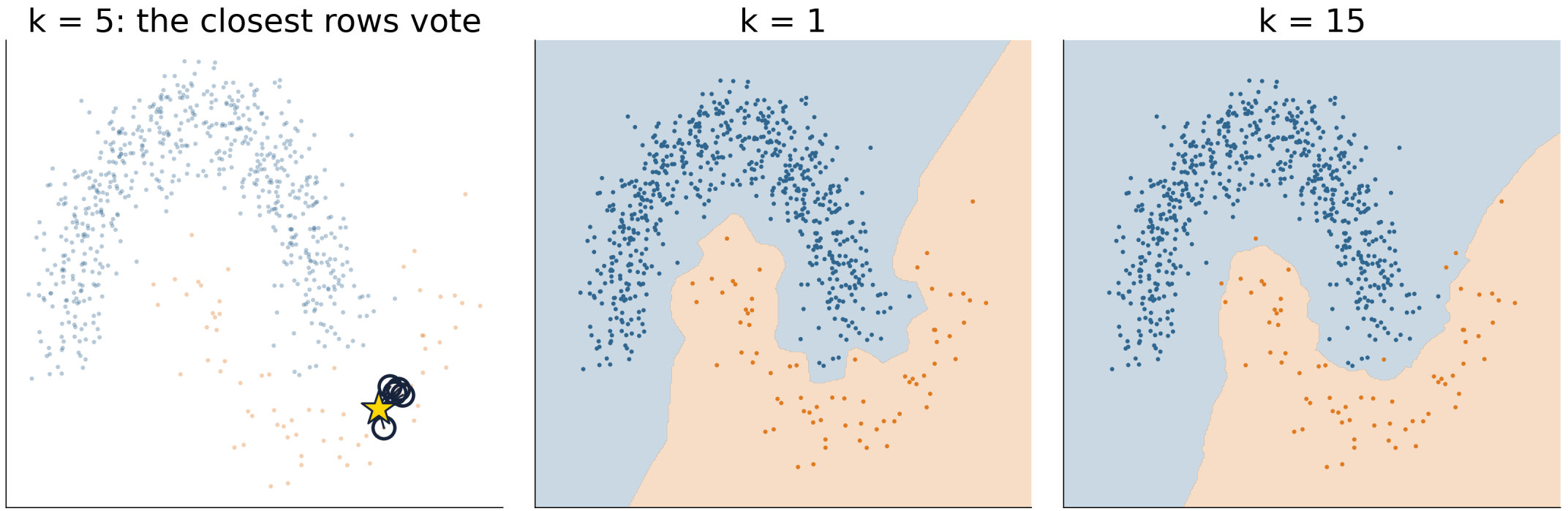
A classifier splits the feature space into regions, one per class. The border between regions is the decision boundary, and it shows what the model has actually learned: a rule that is too simple, about right, or twisted around single points.

Decision tree and random forest



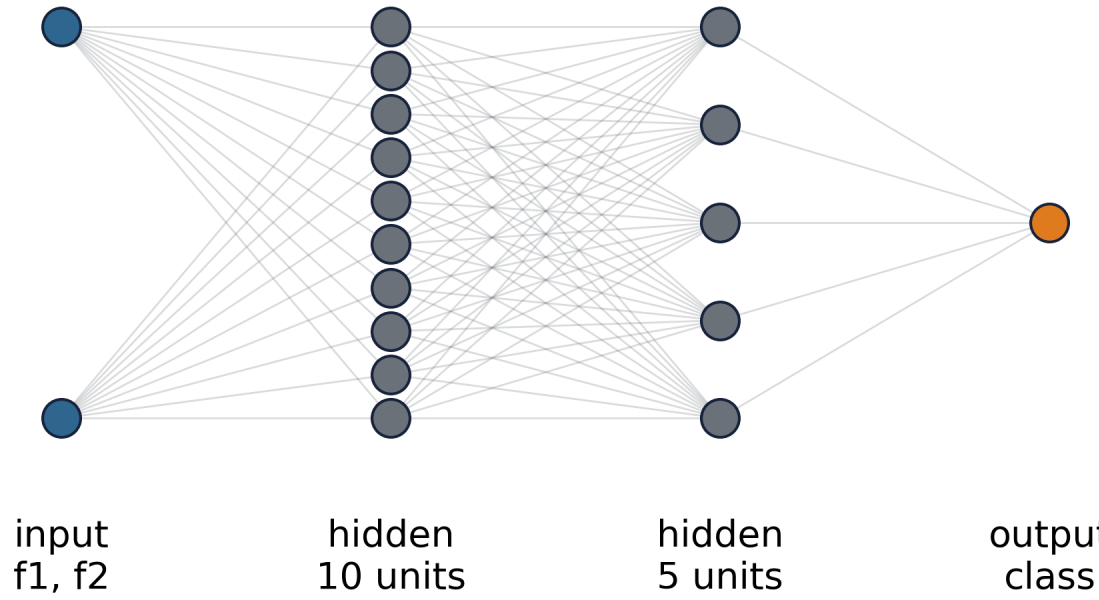
A decision tree asks a sequence of yes/no questions on single features, such as “f1 below a threshold”; the thresholds are its parameters. A random forest trains many trees on random samples of the data and takes the vote of all of them. It does not need scaling.

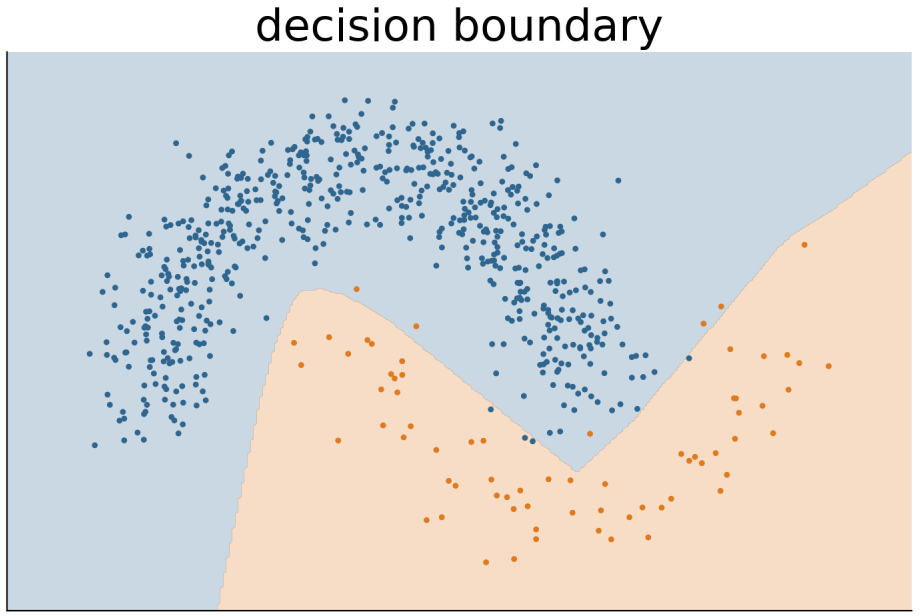
K nearest neighbours (KNN)



KNN keeps all the training rows. To classify a new row it finds the k closest rows and takes the majority vote of their labels. It learns no parameters; the hyperparameter is k. Distances depend on the scale of the features, so the features must be scaled.

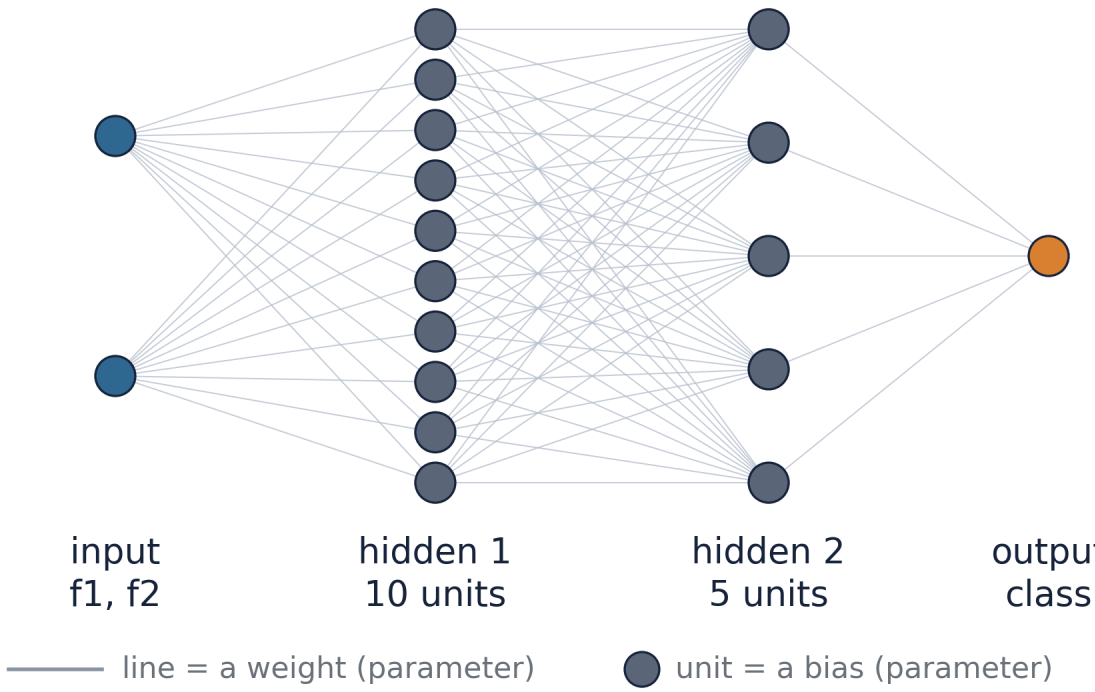
Multi-layer perceptron (MLP)





An MLP is a neural network: layers of units that compute weighted sums of their inputs and pass them through a non-linear function. The weights are the parameters, found by fitting. The hyperparameters are the number and the size of the layers. It needs scaled features.

MLP: parameters and hyperparameters



Hyperparameters: we choose

```
hidden_layer_sizes=(10, 5)
activation='relu'
max_iter=2000
random_state=0
```

Parameters: fit finds them

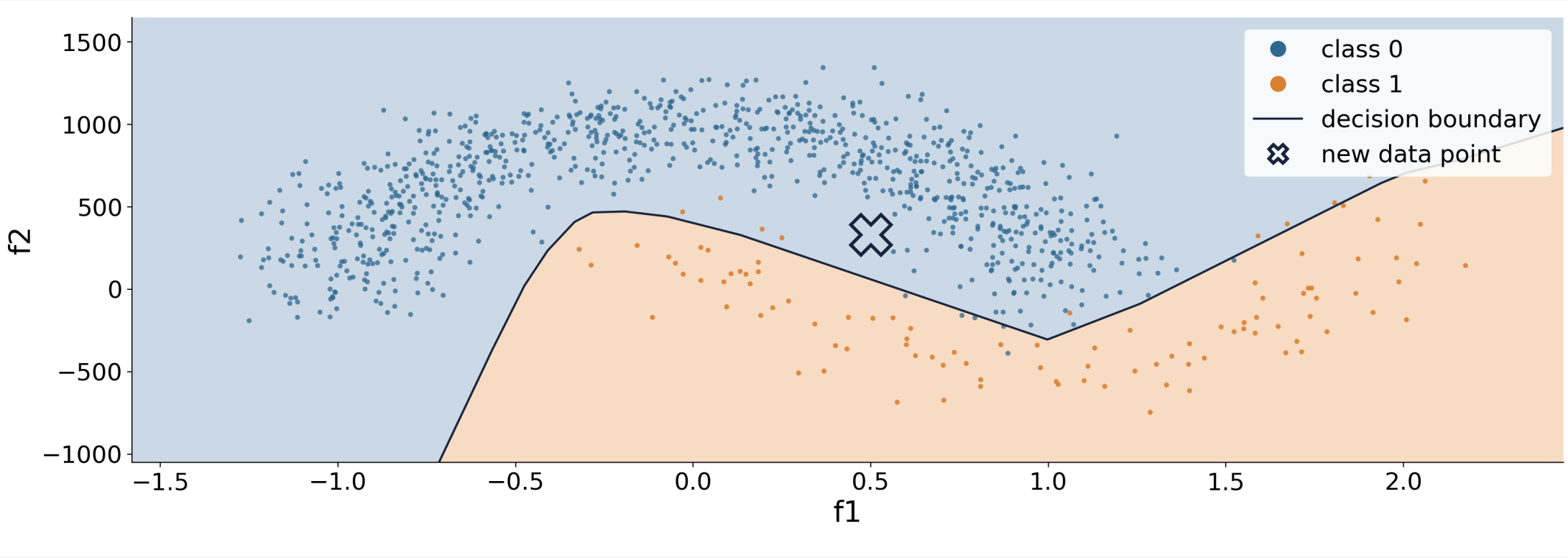
```
input to hidden 1:     2x10 + 10 = 30
hidden 1 to hidden 2:  10x5 + 5 = 55
hidden 2 to output:    5x1 + 1 = 6
total: 30 + 55 + 6 = 91
```

Parameters are the weights (one per line) and the biases (one per unit): fit finds them from the data, 91 numbers here. Hyperparameters are chosen by us before fit: the number of layers, the units in each layer, the activation function, how long to train.

04

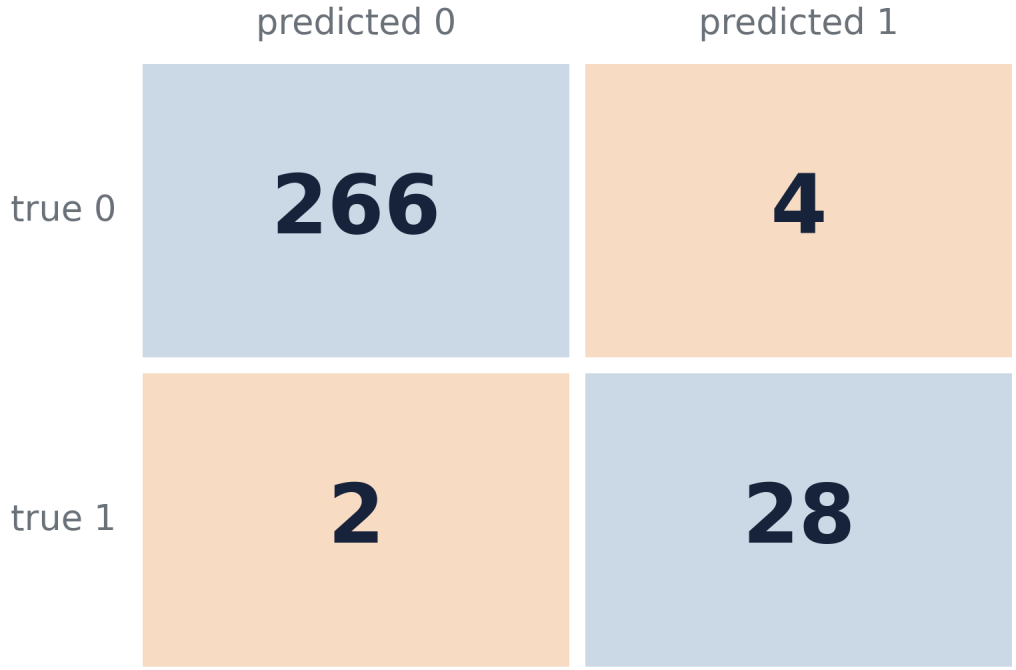
Performance

A new data point



A new row arrives: we know f1 and f2, not the class. The model predicts the class of the region where the point falls.

Confusion matrix, precision, recall, F1, accuracy




```
precision = TP / (TP + FP)
          = 28 / (28 + 4) = 0.88

recall    = TP / (TP + FN)
          = 28 / (28 + 2) = 0.93

F1        = 2TP / (2TP + FP + FN)
          = 56 / (56 + 4 + 2) = 0.90

accuracy  = (TP + TN) / all rows
          = (28 + 266) / 300 = 0.98
```

FP: predicted 1, really 0, a false alarm. FN: predicted 0, really 1, a miss. Precision: of the rows predicted 1, how many are right. Recall: of the rows that are really 1, how many are found. F1: harmonic mean of precision and recall. Accuracy: share of all rows predicted correctly.

05

Reconsider

Minimal ML pipeline

In [1]:

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

df = pd.read_csv('data.csv')
X, y = df[['f1', 'f2']], df['y']

model = RandomForestClassifier()
model.fit(X, y)
predictions = model.predict(X)

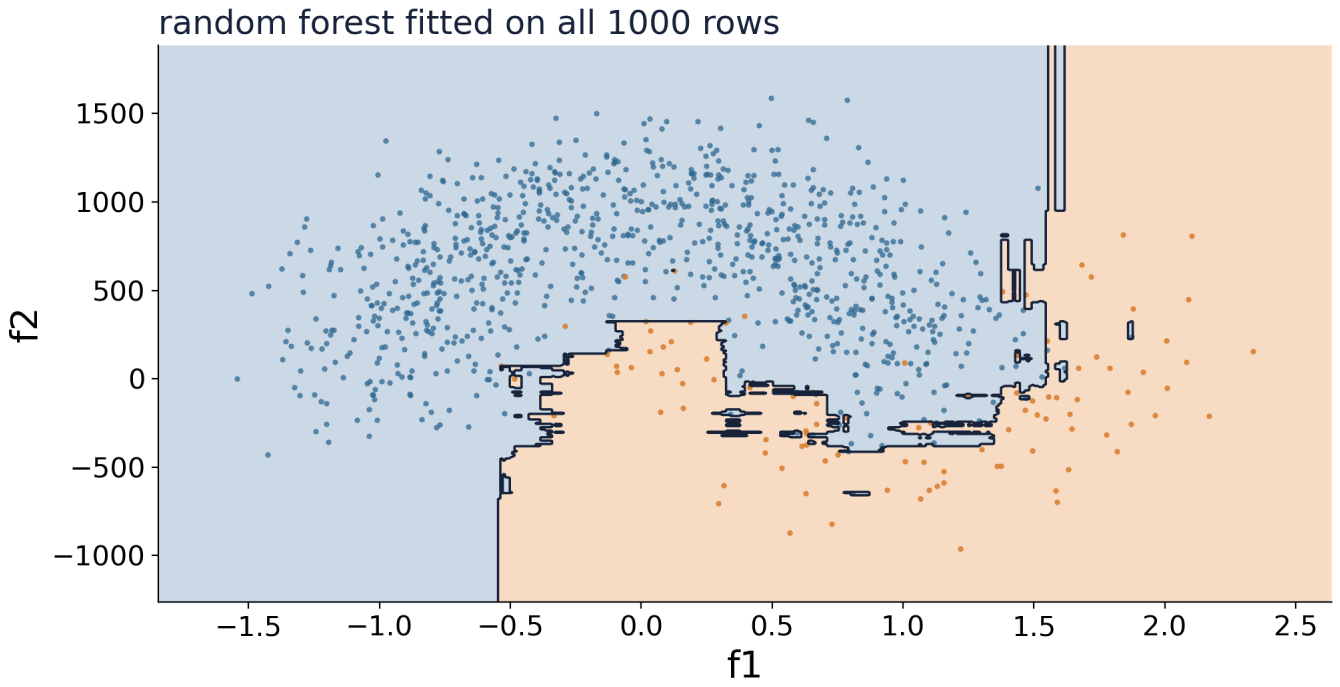
accuracy = accuracy_score(y, predictions)
print(f"Accuracy: {accuracy:.1%}")
```

```
Accuracy: 100.0%
```

06

Pitfalls and practices

Problem: identification is not generalisation



Identification

fit and score on the same rows

`accuracy 1.00`

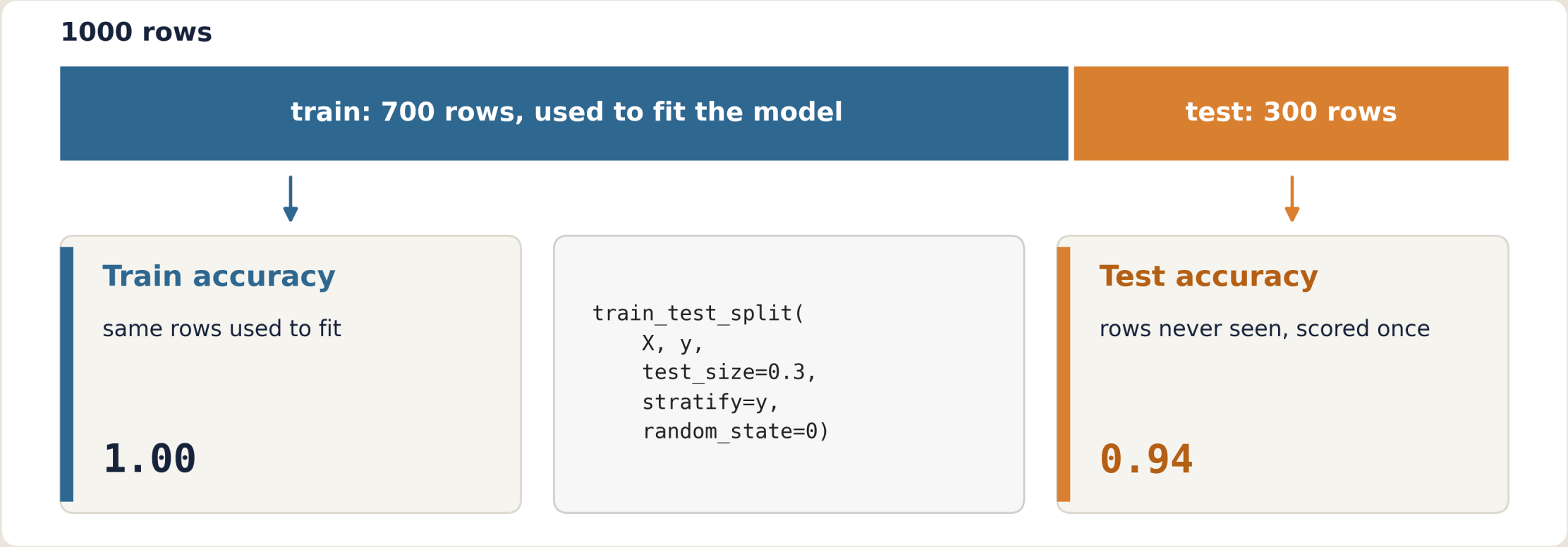
Generalisation

score on rows never seen

`accuracy ?`

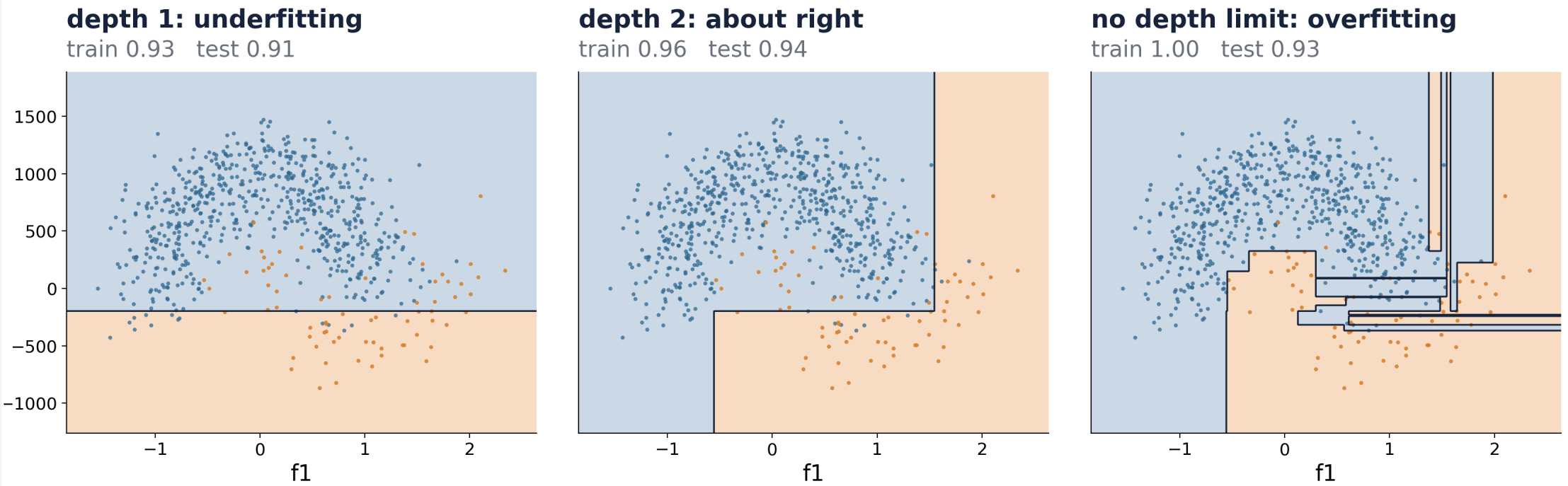
A model fitted and scored on the same rows has only been identified: it can reproduce those rows, 100% here. Generalisation is the score on rows the model has never seen. The first number says nothing about the second.

Fix: train and test split



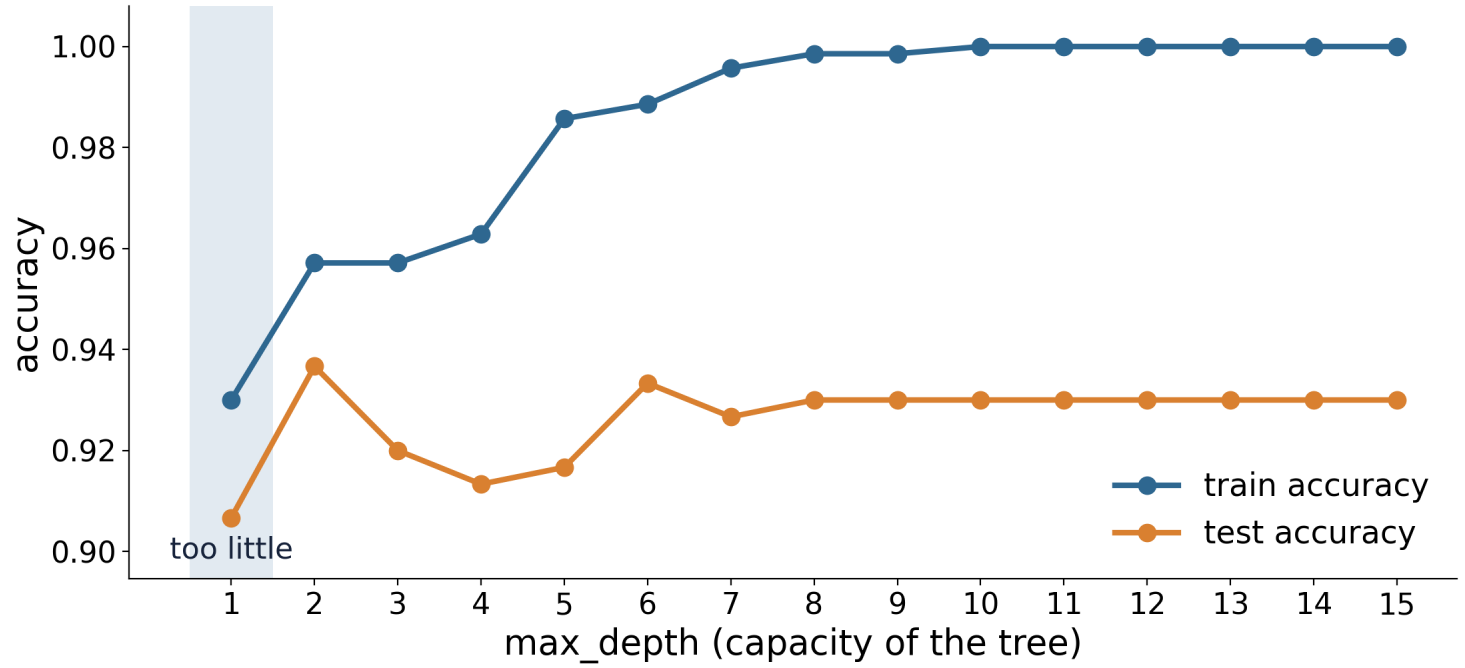
Hide part of the rows before fitting. Fit on the train set, score once on the test set. stratify=y keeps the share of each class equal in the two sets. The test score estimates generalisation: 0.94 here, against 1.00 on the train rows.

Problem: underfitting and overfitting



Capacity is how complicated a rule can be. Too little: the rule misses the shape of the data and is wrong on train and on test (underfitting). Too much: the rule twists around single points and fits the noise of the train set (overfitting).

Fix: control the capacity



Reading the curves

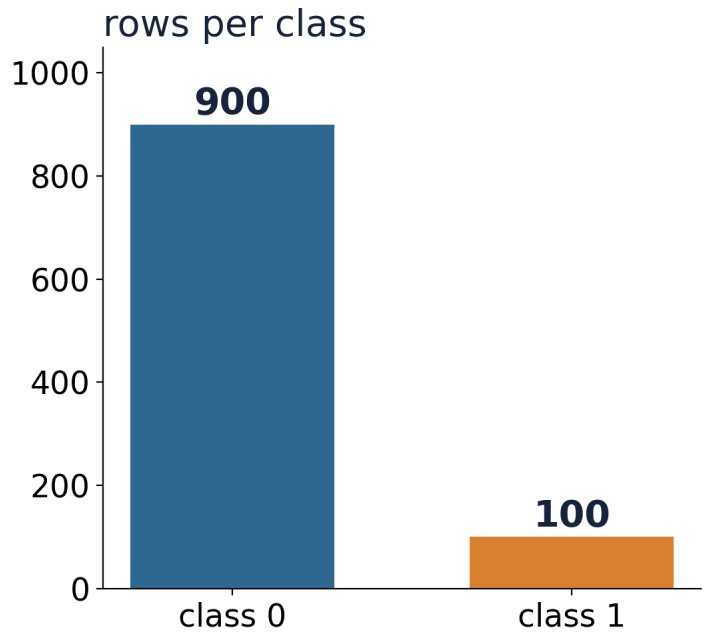
more capacity always
raises the train score

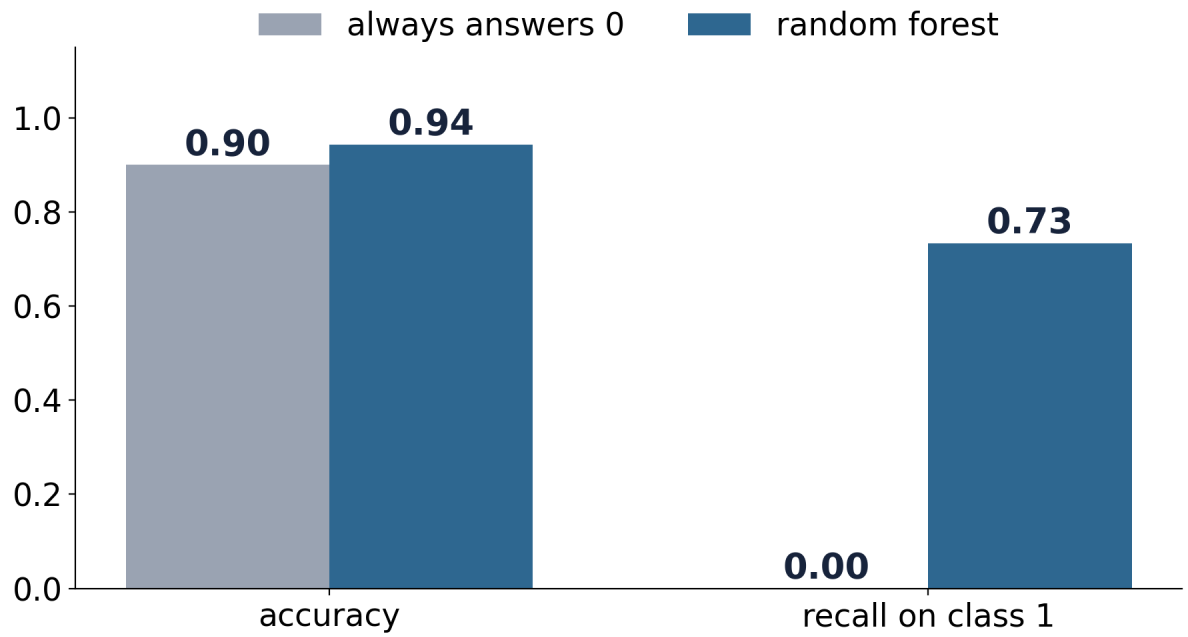
the test score peaks
at depth 2 and then falls

the gap measures overfitting

Limit the capacity with a hyperparameter: max_depth, min_samples_leaf, k. Compare train and test scores: two low scores mean underfitting, a growing gap means overfitting. Choose the simplest model that reaches the best score, here depth 2, using validation data and not the test set.

Problem: unbalanced classes

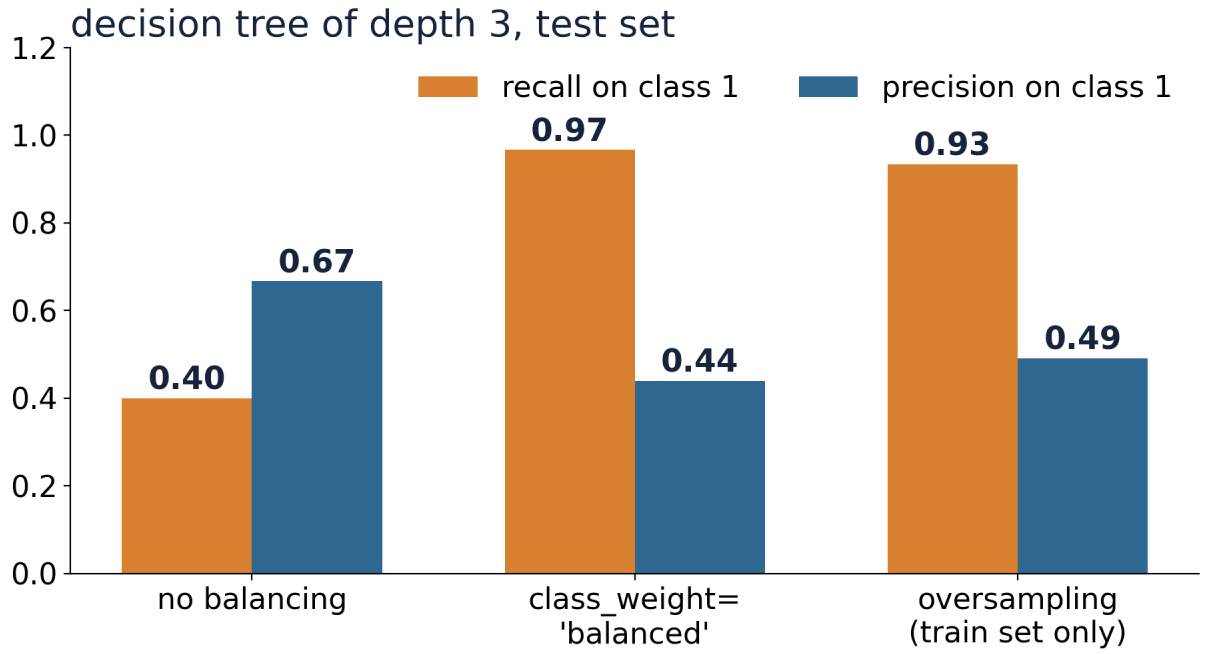




Here 90% of the rows are class 0, so a model that always answers 0 scores 0.90 and finds no row of class 1. Accuracy hides it. That score is the baseline: a model must beat it, and must also be judged by recall on the rare class.

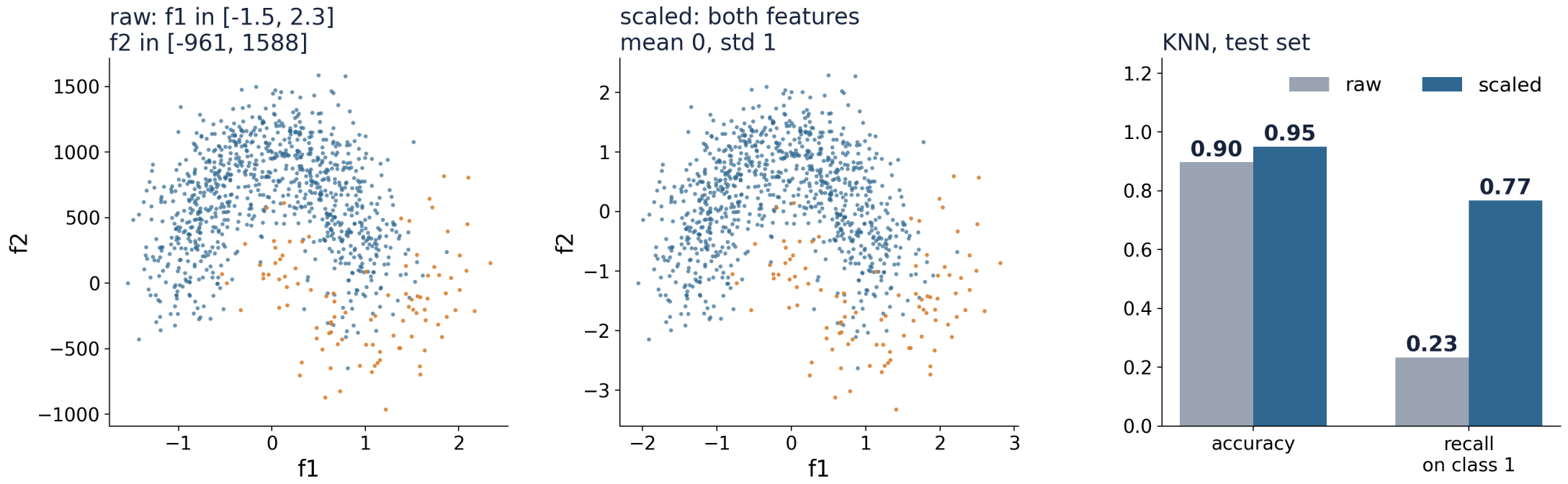
Fix: class balancing





Give the rare class more weight: class_weight='balanced', or oversample it by duplicating rows. Both raise recall on class 1 and lower precision: more cases are found, with more false alarms. Balance the train set only, never the test set.

Problem: features on different scales



f2 is about 1000 times larger than f1. KNN uses distances, so with raw features its neighbours depend almost only on f2 and f1 is ignored. It finds only 23% of the rows of class 1 (recall 0.23), against 77% with scaled features. The MLP also needs scaled features; trees do not.

Fix: rescaling

```
scaler = StandardScaler()
```

Creates the scaler. Each feature becomes z = (x - mean) / std: mean 0 and std 1.

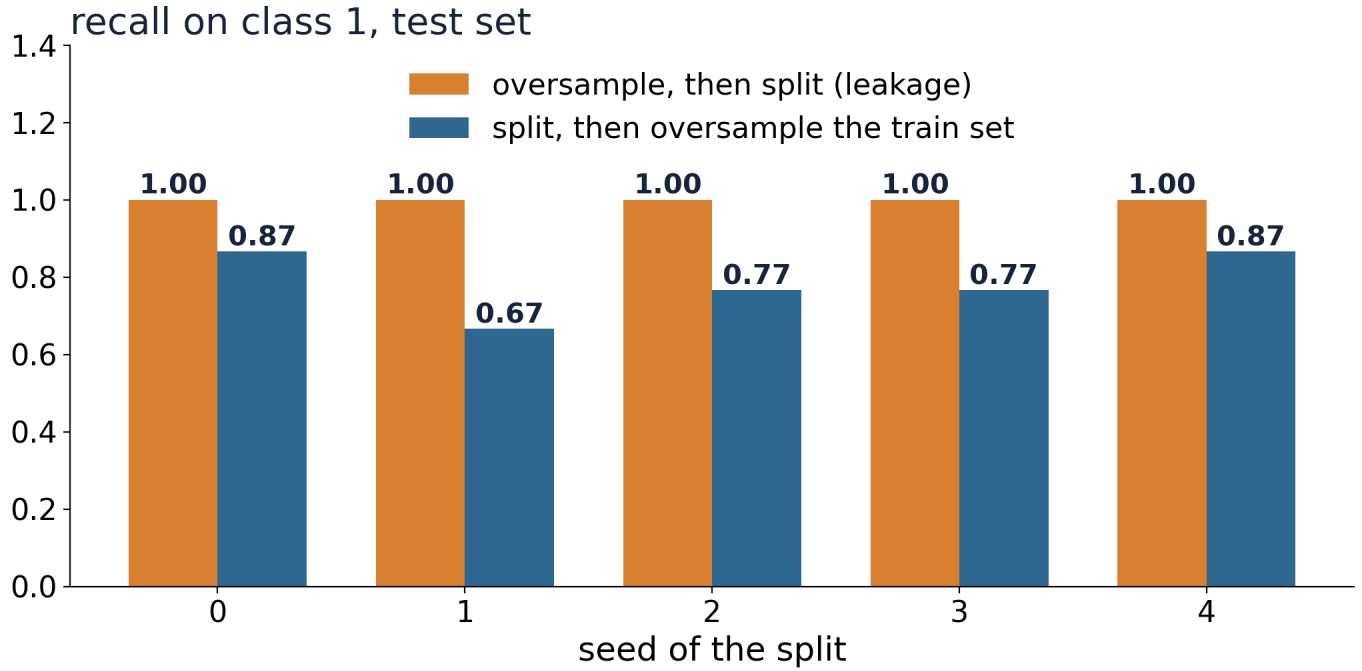
```
X_train_s = scaler.fit_transform(X_train)
```

fit learns the mean and the std of the train set; transform applies them.

```
X_test_s = scaler.transform(X_test)
```

The test set is only transformed, with the train mean and std. It is never fitted.

Problem: data leakage



Why the score is inflated

oversampling copies rows

copies of the same row
land in train and in test

the model is tested on rows
it has already seen

Leakage is any influence of the test set on the training. Example: oversampling before the split puts copies of the same row in train and in test, and recall reaches 1.00 in every split. The score is inflated and will not hold on new data. A scaler or a feature selection fitted on all the rows does the same.

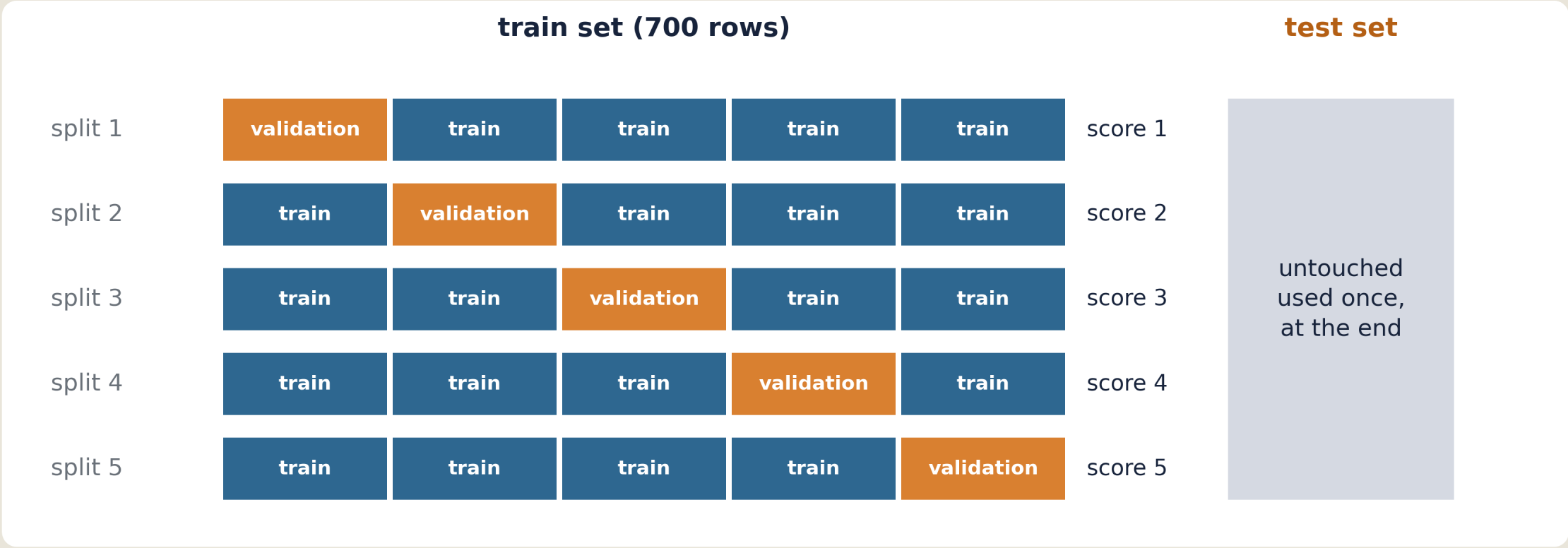
Fix: split first, fit on the train set only

In [ ]:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=0)
pipe = make_pipeline(StandardScaler(), KNeighborsClassifier())
pipe.fit(X_train, y_train)
print(pipe.score(X_test, y_test))
```

The split comes first. The pipeline fits the scaler and the model on the train set only; the test rows are only transformed and scored. The same rule holds for oversampling and for feature selection.

Fix: validation and cross-validation



To choose a hyperparameter we compare several models. Doing it on the test set makes the test set part of the training, and the final score optimistic. Cross-validation splits the train set in k folds: each fold is the validation set once, and the k scores are averaged. The test set stays untouched.

Fix: grid search

k	fold 1	fold 2	fold 3	fold 4	fold 5	mean ± std
1	0.957	0.964	0.964	0.971	0.943	**0.960 ± 0.010**
3	0.943	0.957	0.971	0.971	0.950	**0.959 ± 0.011**
5	0.964	0.957	0.979	0.964	0.950	**0.963 ± 0.009**
9	0.971	0.964	0.979	0.950	0.957	**0.964 ± 0.010**
15	0.964	0.964	0.971	0.950	0.950	**0.960 ± 0.009**
25	0.950	0.950	0.971	0.957	0.950	**0.956 ± 0.008**
51	0.929	0.943	0.943	0.936	0.921	**0.934 ± 0.008**

GridSearchCV

```
grid = GridSearchCV(
    pipe, param_grid, cv=5)
grid.fit(X_train, y_train)
grid.best_params_
grid.score(X_test, y_test)
```

best: k = 9, mean 0.964

refit on the whole train set

test set, used once: 0.947

A grid search is a loop around cross-validation. Each row is a candidate value of the hyperparameter; cross-validation gives it 5 validation scores. The row with the best mean wins, and the model is refitted with that value on the whole train set. The test set is used once, on this model.

Minimal ML CRIME

In [1]:

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

df = pd.read_csv('data.csv')
X, y = df[['f1', 'f2']], df['y']

model = RandomForestClassifier()
model.fit(X, y)
predictions = model.predict(X)

accuracy = accuracy_score(y, predictions)
print(f"Accuracy: {accuracy:.1%}")
```

```
Accuracy: 100.0%
```

ALWAYS DOUBT.

SCIENCE == ASKING QUESTIONS

A result that looks excellent is not a result yet. What was measured? On which data? Compared with what? Could it be this good by accident, or by our own mistake?